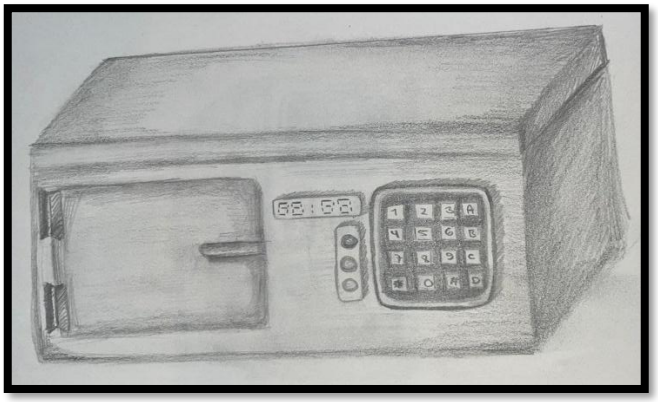
Berufsbildende Schulen Osnabrück Brinkstraße

Arduino-Projekt

„Safe“



Projektdokumentation

Lernfeld 7 „**Cyber-physische Systeme ergänzen**“

Im 2. Schuljahr 2025

Verfasser: Kevin Schumilo, Asad Naeem Saleem

Fachlehrer: Herr Tschorn

Abgabetermin: 08.06.2025

Inhalt

Formulierung des Themas 3
Ressourcen und Ablaufplanung 3
- **Personalplanung** 3
- **Terminplanung** 4

Kostenrechnung 4
Durchführung 5
- **Code** 6
- **Schaltplan** 6

Projektergebnis 7
Anlagen 8

Formulierung des Themas

Im Rahmen dieses Projekts wird ein elektronisch gesteuerter Safe entwickelt, der in Aufbau und Funktionsweise einem Hotelzimmersafe nachempfunden ist. Ziel ist es, mithilfe eines Arduino Uno sowie zusätzlicher Hardwarekomponenten ein funktionstüchtiges System zur sicheren Aufbewahrung von Wertgegenständen zu realisieren.

Der Safe soll sich per PIN-Eingabe öffnen und schließen lassen. Die Bedienung erfolgt durch ein angebrachtes Tastenfeld, über das ein vierstelliger Sicherheitscode eingegeben und mit der Taste „#“ verriegelt werden kann. Um ein unbeabsichtigtes oder unsicheres Verschließen zu verhindern, wird die Eingabe nur dann akzeptiert, wenn die Tür des Safes vollständig geschlossen ist. Sobald die Tür zu ist, wird ein zusätzlicher geschlossener Stromkreis gebildet. Ein digitaler Pin erkennt diesen Zustand und erst dann wird die Eingabe über das Tastenfeld möglich gemacht.

Der Sicherheitscode wird im EEPROM des Arduino gespeichert, um auch bei Stromverlust erhalten zu bleiben. Nur durch Eingabe des korrekten Codes, kann der Safe wieder entsperrt werden. Nach erfolgreicher Entsperrung wird der gespeicherte Code gelöscht – der Safe ist somit für den nächsten Benutzer wieder nutzbar und verschließbar.

Zur Benutzerführung und Rückmeldung wird ein kleines Zifferndisplay eingesetzt, das die Eingaben visuell darstellt. Gleichzeitig sorgen drei LEDs für eine intuitive Statusanzeige: Bei der Verriegelung leuchten zunächst alle drei LEDs gleichzeitig auf. Anschließend erlöschen sie der Reihe nach: zuerst die grüne, dann die gelbe und schließlich die rote LED. Bei einer Entsperrung leuchtet die grüne LED für einige Sekunden auf, während die rote LED gemeinsam mit einem akustischen Signal eine falsche Eingabe darstellt.

Der eigentliche Mechanismus zur Verriegelung der Safe-Tür besteht aus einem Servo-Motor, der einen verstellbaren Riegel betätigt. Dieser ist so verbaut, dass er von außen nicht sichtbar ist und mechanisch verhindert, dass die Tür ohne korrekte Entsperrung geöffnet werden kann.

Die gesamte Steuerlogik – inklusive PIN-Verwaltung, Anzeige- und Verriegelungsmechanismus – wird dabei vom Arduino verwaltet. Dieser wird, neben eines Breadboards, im Inneren des Gehäuses montiert.

Ressourcen und Ablaufplanung

Personalplanung

Das Projekt wurde im Team von zwei Personen umgesetzt. Der überwiegende Teil der Arbeit – insbesondere die Konzeptentwicklung, Softwareprogrammierung, Testphase und

Optimierung – wurde gemeinsam durchgeführt. Die Zusammenarbeit basierte auf kontinuierlichem Austausch, gemeinsamer Problemlösung und gegenseitiger Abstimmung in allen Entwicklungsphasen. Zwei Teilbereiche wurden jedoch klar aufgeteilt, um die Effizienz zu steigern und individuelle Stärken gezielt einzusetzen:

Der Hardwareaufbau wurde fast vollständig von Kevin übernommen. Dazu zählten die Montage aller Komponenten, der Aufbau des Gehäuses, die mechanische Fixierung sowie die vollständige elektrische Verkabelung. Die Dokumentation und Präsentation lagen im Verantwortungsbereich von Asad. Diese Kombination aus gemeinsamer Projektarbeit und gezielter Einzelverantwortung ist in der folgenden Terminplanung wiederzufinden.

Terminplanung

Datum	Dauer (Stunden)	Was	Wer
12.03	0,5	Projektauswahl	Asad & Kevin
19.03	1,5	Projektidee konkret formulieren / Ablaufplan erstellen	Asad & Kevin
23.04	1,5	Recherche zur Funktionsweise der benötigten Bauteile inkl. Adapter	Asad & Kevin
07.05	5	Einzelprüfung aller Bauteile durch Anschluss und Test	Asad & Kevin
11.05	2	Entwurf des Safe-Gehäuses	Asad & Kevin
12.05	2	Beschaffung der Materialien (Hornbach)	Asad & Kevin
14.05	4	Aufbau des Safe-Gehäuses	Kevin
21.05	1,5	Tastenfeld inklusive Adapter integrieren	Kevin
25.05	1,5	Servo-Motor und Buzzer integrieren	Kevin
25.05	1,5	Display integrieren	Kevin
25.05	1,5	LEDs und Taster (Türersatz für Testzwecke) integrieren	Kevin
28.05	5	Speicherung über EEPROM implementieren	Kevin
04.06	3	Abschließende Testphase und Protokollierung	Asad & Kevin
04.06	1,5	Fehlerbehebung	Asad & Kevin
04.06	1	Codestrukturierung und Fertigstellung	Asad & Kevin
11.06	3	Erstellung der Doku Part 1	Asad
18.06	3	Erstellung der Doku Part 2	Asad
18.06	4	Erstellung der PPP	Asad
22.06	1	Finale Abstimmung	Asad & Kevin

Kostenrechnung

Bauteil	Ungefähre Kosten
Arduino UNO	9,50 €
Breadboard	3,45 €
Tastenfeld (4x4)	2,50 €
I2C-Adapter	2,50€
Servomotor	3,50 €
Aktiver Buzzer	1,50 €
Display TM1637 (4 Digit 7-Segment Display)	2,30 €
2x Widerstände	0,05 €
Batterie + Adapter	3,75 €
Ampel LED	2,99 €
25x Jumper-Kabel	2,50 €
Safe-Gehäuse (Kassenbon im Anhang)	30,60€
Total	65,14€

Durchführung

Zu Beginn werden alle, aus der Projektideenfindung, notwenigen Komponenten beschafft und zusammengeführt. Daraufhin wird die Implementierung je Komponente recherchiert, Bibliotheken rausgesucht und getestet.

Im Rahmen dieses Projekts wurde zunächst gemeinsam im Team die Idee für den elektronisch gesteuerten Safe konkretisiert. Dabei legten wir die grundlegenden Funktionen und Anforderungen fest: Die Tür sollte sich ausschließlich per PIN-Eingabe über ein extern angebrachtes Tastenfeld öffnen und schließen lassen. Es musste sichergestellt werden, dass die Eingabe nur erfolgen kann, wenn die Safe-Tür vollständig geschlossen ist. Für die Umsetzung wurde ein Ablaufplan erstellt, der die einzelnen Arbeitsschritte und Verantwortlichkeiten klar definierte.

Nach der Planungsphase begann die Recherche zu den benötigten Komponenten. Insbesondere wurden Funktionsweise und Anschlussmöglichkeiten des Arduino Uno, des Tastenfelds, des Servo-Motors, des EEPROM-Speichers sowie der LED-Anzeigen und des Buzzers untersucht. Um spätere Probleme zu vermeiden, testeten wir alle Bauteile zunächst einzeln an einem Testaufbau.

Auf Basis der gewonnenen Erkenntnisse wurden der Schaltplan erstellt, welche die Verkabelung der Komponenten darstellt. Im Anschluss begannen wir mit dem Aufbau der Hardware. Da kein fertiges Safe-Gehäuse zur Verfügung stand, kauften wir zunächst ein passendes Gehäuse und alle erforderlichen Materialien. Dabei nutzen wir unsere Entwurfsskizzen für den Aufbau. Die Bauteile wurden sorgfältig im Safe-Gehäuse montiert und elektrisch verbunden. Der Servo-Motor wurde so eingebaut, dass er den Verriegelungsmechanismus verdeckt und mechanisch eine Öffnung ohne korrekte PIN-Eingabe verhindert.

Parallel zur Hardwareentwicklung wurde die Software implementiert. Die Programmierung erfolgte sequenziell, sodass die einzelnen Funktionen – etwa das Auslesen der PIN-Eingaben, die Steuerung der LEDs und des Servomotors sowie die Verwaltung des Sicherheitscodes im EEPROM – separat entwickelt und getestet werden konnten. Dies erleichterten die Fehlersuche und die schrittweise Integration aller Systembestandteile. Die Eingabe des vierstelligen Codes wird erst dann akzeptiert, wenn der Stromkreis über den Metallkontakt an der Tür geschlossen wird, wodurch ein unbeabsichtigtes Ver- oder Entriegeln verhindert wird. Nach Eingabe des korrekten Codes wird der Servomotor angesteuert, der die Tür entriegelt, und der Code im EEPROM gelöscht, sodass der Safe für den nächsten Benutzer wieder nutzbar ist.

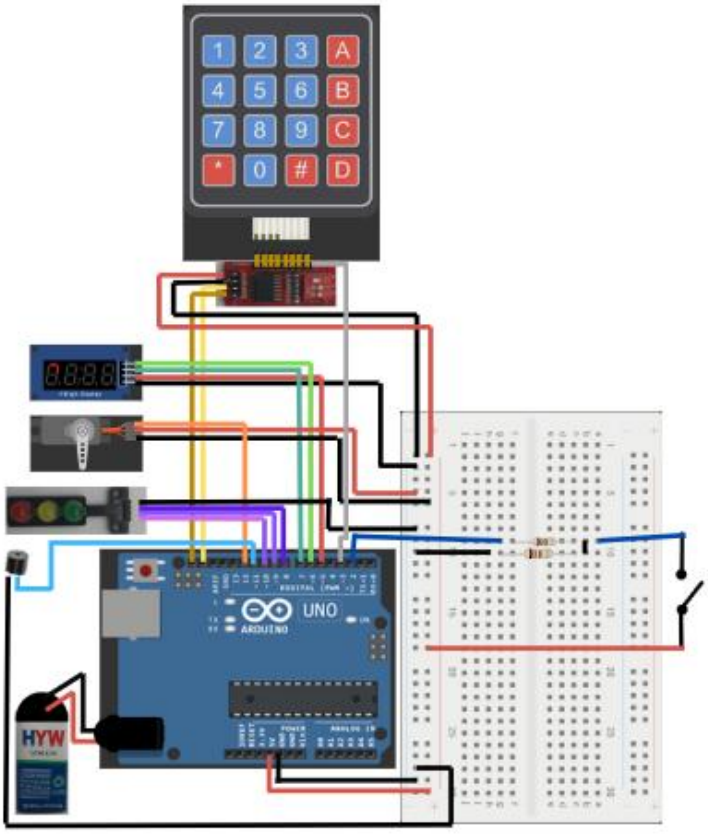
Abschließend wurde eine umfassende Projektdokumentation erstellt, die alle Entwicklungsschritte, Schaltpläne, Programmcode sowie Testergebnisse enthält. In diesem Zusammenhang erfolgte die Vorbereitung der Präsentation und Live-Demonstration des elektronischen Safes, um die Funktionalität und den Projektverlauf anschaulich darzustellen.

Code

Der Code wurde folgendermaßen aufgebaut. Zu Beginn werden die Bibliotheken eingebunden. Danach wird ein Struct-Objekt deklariert, für das spätere Speichern des PINs. Dem I2C - Keypad wird mittels dem I2C-Adapter eine physische Adresse zugewiesen. Dann folgt die Definition respektive Deklaration der Variablen, der LED-Pins, sowie der Pins der übrigen Komponenten. Als nächstes werden Funktionen definiert für die Passwort-Speicherung, dem Abrufen des Passwortes über vom EEPROM-Speicher, Passworteingabevergleich, Löschen des Passwortes, dem Start des Energiesparmodus.

In der Setup-Methode wird die Pin-Belegung gesetzt, dann werden die Komponenten initiiert. Der Loop-Teil beinhaltet keine direkten Delays, sondern eine asynchrone Implementierung für die einzelnen Komponenten (Funktion milis). Durch einen großen if-Block, werden die Eingaben validiert und dementsprechend behandelt.

Schaltplan



Projektergebnis

Zur Sicherstellung der Funktionsfähigkeit und Qualität wurden gezielte Testfälle durchgeführt, die sowohl Einzelfunktionen als auch das Gesamtsystem prüften. Somit konnte auch ein direkter „Soll-Ist-Vergleich“ stattfinden, mit anschließender Behebung:

Test	Erwartung	Ergebnis	Anpassung
Korrekte PIN-Eingabe bei geschlossener Tür	Servo öffnet, grüne LED leuchtet, PIN wird gelöscht	Funktioniert, jedoch leichte Verzögerung beim Öffnen	Servoposition behoben
Falsche PIN-Eingabe bei geschlossener Tür	Rote LED leuchtet, Signalton, keine Öffnung	Erfolgreich	/
Verhalten bei Stromausfall	PIN bleibt im EEPROM gespeichert	Erfolgreich	/
Verriegelung nach erneutem Schließen der Tür	Servo verriegelt, Alle LEDs leuchten und gehen nacheinander aus.	Funktioniert, jedoch war die Reaktionszeit der LEDS zu schnell	Einfügen eines kleinen Delays

Für eine Weiterentwicklung des Projekts ergeben sich mehrere interessante Ansätze. Ersteres wäre ein Schutzmechanismus gegen ***Brute-Force-Angriffe***. Aktuell ist es möglich, unbegrenzt viele PIN-Versuche durchzuführen, was eine potenzielle Schwachstelle darstellt. Eine Sperrfunktion, die den Safe nach mehreren falschen Eingaben für eine gewisse Zeit (z. B. 30 Sekunden) blockiert, würde die Sicherheit deutlich erhöhen und das System robuster gegen Manipulationsversuche machen. Des Weiteren wäre ein **Notfall-Reset-Funktion** sehr interessant. Ein häufiger Schwachpunkt in PIN-basierten Systemen ist das Vergessen des Codes durch den Benutzer. Ein sinnvoller Zusatz wäre daher eine versteckte „Reset“-Möglichkeit, z. B. durch einen internen Schalter oder eine definierte Tastenkombination, mit der sich der Safe zurücksetzen lässt. Damit kann der Benutzer den Safe wieder programmieren, ohne das Gehäuse öffnen zu müssen – ideal für versehentliche Sperrungen. Die Letzte Erweiterung handelt von einer **Statusanzeige über eine Weboberfläche**, welche Daten zu Punkten wie: offen/geschlossen, letzte Eingabezeit, Sperrstatus nach Fehlversuchen usw. darstellen könnte.

Insgesamt wurde das Ziel eines funktionalen elektronischen Safes erreicht, jedoch mit realistischen Herausforderungen in Bezug auf Zuverlässigkeit, Zeitmanagement und Feineinstellungen. Das Projekt bleibt technisch ausbaufähig und bietet vielfältige Erweiterungsmöglichkeiten.

Anlagen

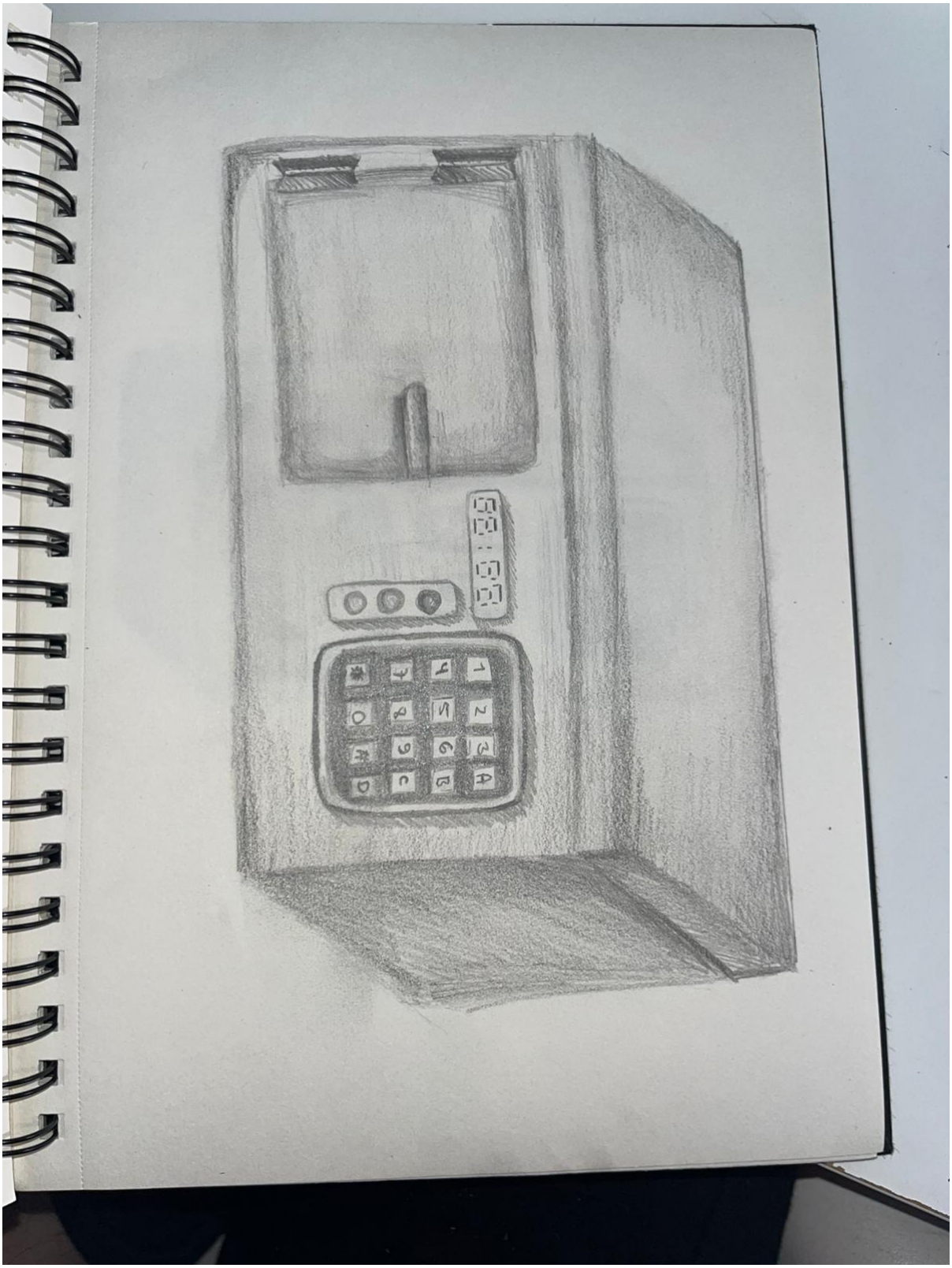
HORNBACH
Es gibt immer was zu tun.

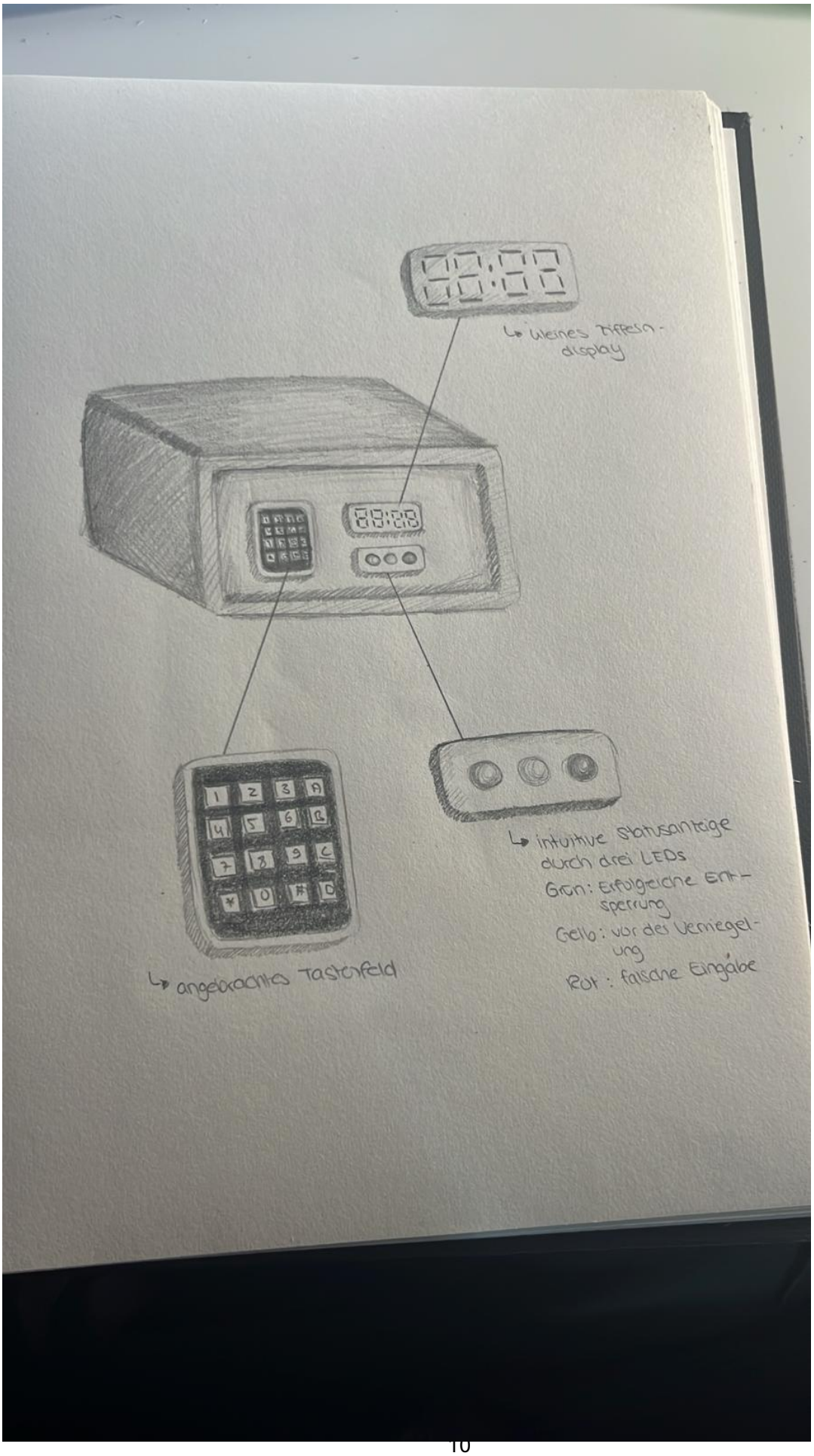
HORNBACH Baumarkt AG
Filiale 611
Hannoversche Str.111
49084 Osnabrück
Tel. 0541 58288-0
WEEE-Reg.-Nr. DE 39697378
Steuernummer: 24/652/3000/5

ART/EAN 5906210008230		
1 Stück x	3,95	
Holz-Deckel 40x30		3,95 1
ART/EAN 4306517946357		
6 Stück x	1,10	
Verstellwinkel		6,60 1
ART/EAN 4004338345419		
1 Stück x	1,45	
Scharnier, schmal, m		1,45 1
ART/EAN 4004338342838		
1 Stück x	1,10	
Verstellwinkel		1,10 1
ART/EAN 4004338343071		
7 Stück x	0,40	
Pk Flachverbinder		2,80 1
ART/EAN 4306517946067		
1 Stück x	0,80	
Pk Verstellwinkel		0,80 1
ART/EAN 5906210008025		
1 Stück x	6,95	
Holz-Allzweckkiste		6,95 1
ART/EAN 4007622047426		
1 Stück x	1,95	
Balkonbrett schlicht		1,95 1
ART/EAN 2007002606344		
1,000 LeistE x	5,00	
Zuschnittsrest Platt		5,00 1

SUMME [20] EUR 30,60

GEGEBEN VISA EUR 30,60


1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D


↳ kleines Ziffern-display
↳ intuitive Statusanzeige durch drei LEDs
Grün: Erfolgreiche Entsperrung
Gelb: vor der Verriegelung
Rot: falsche Eingabe
↳ angebrachtes Tastenfeld

Code

```
#include "TM1637.h"

#include "LowPower.h"

#include <Servo.h>

#include <I2CKeyPad.h>

#include <Wire.h>

#include <EEPROM.h>


// Password Object

struct Password {

  bool set;

  int8_t secret[4];

};


// Konstanten definieren

#define KeyPad_I2C_ADDR 0x21


// Variablen definieren

int sequenceNum = 0;

bool keyPressed = false; // um ein Debounce zu verhindern

bool turnedOn = true; // nach verlassen des Energiesparmodus, sollen einmalig einige
Einstellungen ausgeführt werden

bool doorOpen = true; // um einmalig die Energiesparmodus-Einstelllugen festzulegen

bool passwordIsSet = false; // gibt an, ob ein Passwort im Speicher hinterlegt ist

int8_t passwordTemp[4]; // hält das Passwort aus dem Speicher, um damit zu arbeiten

unsigned long onTime = 0; // Zeit in Millisekunden, seit dem der Arduino an ist

unsigned long onTime_p = 0; // letzte Zeit in Millisec., in der ein Input geschah

unsigned long dispTime = 0; // Zeit in Millisekunden fuer das Display

unsigned long dispTime_p  = 0; // abgleich der verstrichenen Zeit

unsigned long keyTime = 0; // Zeit in Millisekunden fuer das Keypad

unsigned long keyTime_p  = 0; // abgleich der verstrichenen Zeit

unsigned long buzTime = 0;// Zeit in Millisekunden fuer den Buzzer

unsigned long buzTime_p  = 0; // abgleich der verstrichenen Zeit
```

```
bool wrongInput = false; // Variable fuer falsche Eingabe
unsigned long buzTimeWrongInput = 0; // Zeit in Millisekunden fuer den Buzzer bei falscher Eingabe
unsigned long buzTimeWrongInput_p = 0; // abgleich der verstrichenen Zeit
bool unlocked = false; // Variable, falls der Safe entsperrt wurde
bool locked = false;  // Variable, falls der Safe gesperrt wurde
int afkTime = 20000; // Zeit in Millisec., nach der Energiesparmodus beginnt
bool tryAgain = false; // sobald die Eingabe von Anfang beginnen soll
int degree = 0; // Servomotor-Riegel Winkelstellung in Grad

// LED Pin Belegung
unsigned long openingLED = 0;
unsigned long openingLED_p = 0;
unsigned long wrongPinLED = 0;
unsigned long wrongPinLED_p = 0;
unsigned long closingLED = 0;
unsigned long closingLED_p = 0;

// PINs definieren
const int ServoPin = 12;
const int BUZ = 11;
const int Disp = 5;
const int Door = 2; // Interrupt fähiger PIN
const int keyPadInt = 3; // Interrupt fähiger PIN
const int greenLED = 8;
const int redLED = 9;
const int yellowLED = 10;

// I2C Keypad
I2CKeyPad keypad(KeyPad_I2C_ADDR);
char keyMap[19] = "123 456 789  0# NF";

// 7 Segment 4 Digit Display
```

```
int8_t pinCode[4];
const int Disp_CLK = 6;
const int Disp_DIO = 7;
TM1637 tm1637(Disp_CLK, Disp_DIO);

// Servo Motor (Türriegel)
Servo Servomotor;

void wakeUp()
{
  // Just a handler for the pin interrupt.
}

// Speichert das Passwort in den EEPROM Speicher
void setPassword(int8_t* secret) {
  Password password;
  password.set = true;
  for (int i = 0; i < 4; i++) {
    password.secret[i] = secret[i];
  }

  EEPROM.put(0, password);
}

// Holt sich die Info aus dem Speicher, ob dort ein Passwort gespeichert ist.
bool getPassword(int8_t* tempPassword) {
  Password myPassword;
  EEPROM.get(0, myPassword);

  if (!myPassword.set) {
    return false;
  }
```

```
  for (int i = 0; i < 4; i++) {
    tempPassword[i] = myPassword.secret[i];
  }

  return true;
}

// Speicher löschen bzw. überschreibt jede Stelle mit Nullen
void clearStorage() {
  for (int i = 0 ; i < EEPROM.length() ; i++) {
    EEPROM.write(i, 0);
  }
}

// Vergleiche das gespeicherte Passwort mit der Eingabe und gebe ein true/false zu-
rück
bool inputMatchesPassword(int8_t* input, int8_t* password) {
  for (int i = 0; i < 4; i++) {
    if (input[i] != password[i]) {
      return false;
    }
  }

  return true;
}

void applyPowerSaving() {
  digitalWrite(Disp, LOW); // Display ausschalten (Energiesparmaßnahme)

  // Enter power down state with ADC and BOD module disabled.
  // Wake up when wake up pin is low.
  doorOpen = false;
  LowPower.powerDown(SLEEP_FOREVER, ADC_OFF, BOD_OFF);
```

```
  // der Interrupt führt den Code hier weiter aus..
  turnedOn = true; // um das Aufwachen zu registrieren
}

// Nachdem der Arduino den Energiesparmodus verlässt
void setRunningParameter() {
  onTime_p = onTime;
  turnedOn = false;
  detachInterrupt(digitalPinToInterrupt(Door)); // Vorher gesetzten Interrupt löschen
  detachInterrupt(digitalPinToInterrupt(keyPadInt)); // Vorher gesetzten Interrupt lö-
schen
  memset(pinCode, 0, sizeof(pinCode)); // um das Display mit Nullen füllen -> erneute
Eingabe
  digitalWrite(Disp, HIGH);
  doorOpen = true;
  sequenceNum = 0;
}

void setup() {
  // PIN-Modi definieren
  pinMode(BUZ, OUTPUT);
  pinMode(Door, INPUT);
  pinMode(Disp, OUTPUT);
  pinMode(keyPadInt, INPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);

  // Servomotor
  Servomotor.attach(ServoPin);

  // Keypad
```

```
  keypad.loadKeyMap(keyMap);

  // Display
  tm1637.init();
  tm1637.set(BRIGHT_TYPICAL);//BRIGHT_TYPICAL = 2,BRIGHT_DARKEST =
0,BRIGHTEST = 7;

  Serial.begin(9600);
  keypad.begin();
  Wire.begin();
  delay(1000);

  //clearStorage(); // Falls der Safe gesperrt wurde und der Pin vergessen wird

  // Rückstellung bzw. Kallibrierung des Servomotor-Stellung
  if (!getPassword(passwordTemp)) {
    Servomotor.write(degree);
  }
}

void loop() {
  onTime = millis();
  buzTime = millis();
  openingLED = millis();
  wrongPinLED = millis();
  closingLED = millis();
  buzTimeWrongInput = millis();

  // Bei falscher Pin Eingabe
  if (wrongInput) {
    if (buzTimeWrongInput - buzTimeWrongInput_p >= 1050) {
      digitalWrite(BUZ, LOW);
      digitalWrite(redLED, LOW);
```

```
    buzTimeWrongInput_p = buzTimeWrongInput;
    wrongInput = false;
  }
}

// Nach Tastendruck soll der Buzzer nach 35 Millisekunden aus gehen
if (!wrongInput && keyPressed && (buzTime - buzTime_p >= 35)) {
  buzTime_p = buzTime;
  digitalWrite(BUZ, LOW);
  keyPressed = false;
}

// Falls der Safe entsperrt wurde
if (unlocked && (openingLED - openingLED_p >= 300)) {
  memset(pinCode, 0, sizeof(pinCode)); // um das Display mit Nullen füllen
  if (openingLED - openingLED_p >= 700) {
    unlocked = false;
    clearStorage(); // ..braucht Zeit!
    digitalWrite(greenLED, LOW);
  }
}

if (locked && closingLED - closingLED_p >= 700) {
  if (closingLED - closingLED_p >= 900) {
    digitalWrite(greenLED, LOW);
  }

  if (closingLED - closingLED_p >= 1100) {
    digitalWrite(yellowLED, LOW);
  }

  if (closingLED - closingLED_p >= 1300) {
    digitalWrite(redLED, LOW);
```

```
    locked = false;
  }
}

// Zuruecksetzen der Zahlen, für erneute Eingabe bei vorhandenem Passwort
if (tryAgain && !wrongInput && !unlocked && !locked) {
  sequenceNum = 0;
  tryAgain = false;

  memset(pinCode, 0, sizeof(pinCode)); // um das Display mit Nullen füllen
}



int doorClosed = !digitalRead(Door);
if (doorClosed) {
  tm1637.display(pinCode); // Zahl auf dem Bildschirm ausgeben

  // Nach der 'away-from-keyboard' Zeit, soll in den Energiesparmodus gewechselt
werden
  if (onTime - onTime_p >= afkTime) {
    attachInterrupt(digitalPinToInterrupt(keyPadInt), wakeUp, CHANGE); // Interrupt de-
finieren
    attachInterrupt(digitalPinToInterrupt(Door), wakeUp, CHANGE); // Interrupt definie-
ren
    applyPowerSaving();
  }

  // Wenn der Arduino direkt zuvor im Energiesparmodus oder ausgeschaltet war
  if (turnedOn) {
    setRunningParameter();
  }

  // Hier wird die Eingabe registriert und bearbeitet
  keyTime = millis();
```

```
  char keyAsChar = keypad.getChar();
  if ((keyTime - keyTime_p >= 650) && keyAsChar != ' ' && keyAsChar != 'F') { // Wenn
die deaktivierten Tasten gedrückt werden oder zwei Tasten gleichzeitig.
    if (keyAsChar != 'N') {

      onTime_p = onTime; // resetten der Zeit, bis der Energiesparmodus eintritt
      keyTime_p = keyTime; // resetten der Zeit, bis ein Tastendruck möglich ist

      digitalWrite(BUZ, HIGH);

      if (keyAsChar != '#') {
        char keyAsAscii = keyAsChar - '0'; //Um bei der Umwandlung von char zu int die
korrekten Zahlen zu erhalten

        if (sequenceNum < 4) {
          pinCode[sequenceNum] = (int8_t)keyAsAscii;
          sequenceNum++;
          tm1637.display(pinCode); // Zahl auf dem Bildschirm ausgeben
        }

        if (sequenceNum == 4) {
          if (!getPassword(passwordTemp)) {
            Servomotor.write(degree);

          } else {
            if (inputMatchesPassword(pinCode, passwordTemp)) {
              Serial.println("richtiges Passwort!");
              digitalWrite(greenLED, HIGH);
              openingLED_p = openingLED;
              Servomotor.write(degree);
              unlocked = true;
```

```
            } else {
              Serial.println("FALSCH!!");
              digitalWrite(redLED, HIGH);
              digitalWrite(BUZ, HIGH);
              wrongPinLED_p = wrongPinLED;
              buzTimeWrongInput_p = buzTimeWrongInput;
              wrongInput = true;
            }

            tryAgain = true;
          }
        }
      } else if (keyAsChar == '#' && sequenceNum == 4 && !getPassword(pass-
wordTemp)) {
        digitalWrite(yellowLED, HIGH);
        digitalWrite(redLED, HIGH);
        digitalWrite(greenLED, HIGH);
        closingLED_p = closingLED;
        Servomotor.write(degree + 92); // Schliessen
        setPassword(pinCode);
        // attachInterrupt(digitalPinToInterrupt(keyPadInt), wakeUp, CHANGE); // Inter-
rupt definieren
        tryAgain = true;
        locked = true;
      }

      keyPressed = true;
    }
  }
} else if (doorOpen) {
  attachInterrupt(digitalPinToInterrupt(Door), wakeUp, CHANGE); // Interrupt definie-
ren
  applyPowerSaving();
}
```

```
}
```